

CubeMX 配置

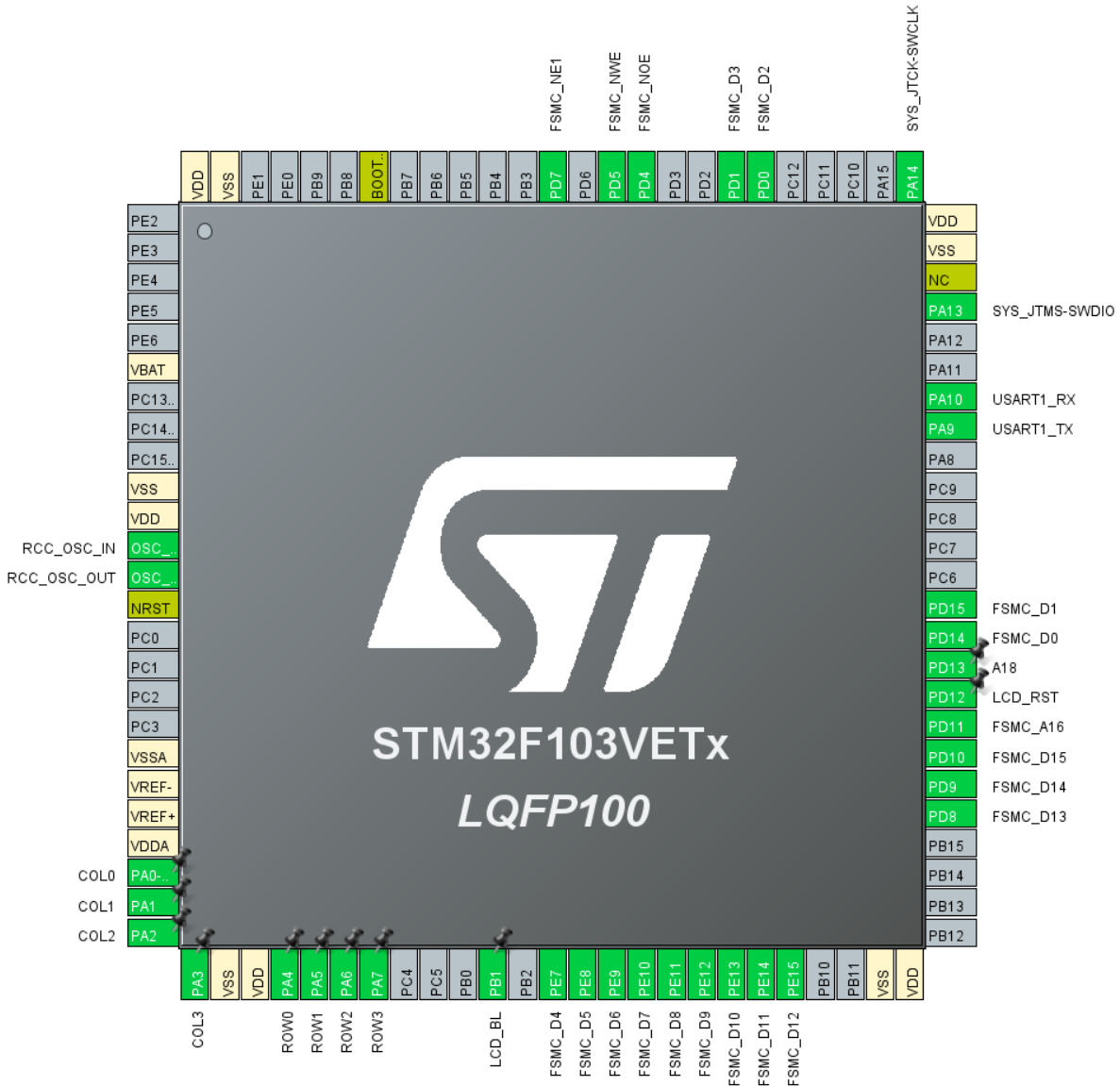


图 3-3 STM32F103VET6 引脚分配与外设配置

GPIO FSMC RCC SYS USART NVIC							
Search Signals							
Search (Ctrl+F)						☐ Show only Modified Pins	
Pin Name	Signal on Pin	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PA0-WKUP	n/a	Low	Output Pu...	No pull-up ...	Low	COL0	☒
PA1	n/a	Low	Output Pu...	No pull-up ...	Low	COL1	☒
PA2	n/a	Low	Output Pu...	No pull-up ...	Low	COL2	☒
PA3	n/a	Low	Output Pu...	No pull-up ...	Low	COL3	☒
PA4	n/a	n/a	External In...	Pull-down	n/a	ROW0	☒

图 3-4 4×4 矩阵键盘 GPIO 引脚配置

CubeMX 配置界面中仅保留系统自动生成的默认任务。电梯控制任务、串口解析任务、按键输入任务和状态显示任务均在 freertos.c 的 MX_FREERTOS_Init()函数中使用 xTaskCreate()动态创建。两个消息队列和两个互斥量同样通过 FreeRTOS 原生 API 手动创建。

```
215 ret = xTaskCreate(ElevatorTask,
216                  "Elevator",
217                  384,
218                  NULL,
219                  tskIDLE_PRIORITY + 3,
220                  &elevatorTaskHandle);
221
222 if (ret == pdPASS)
223     printf("ElevatorTask create OK\r\n");
224 else
225     printf("ElevatorTask create FAIL\r\n");
226
227 ret = xTaskCreate(UartParseTask,
228                  "UartParse",
229                  256,
230                  NULL,
231                  tskIDLE_PRIORITY + 2,
232                  &uartParseTaskHandle);
233
234 if (ret == pdPASS)
235     printf("UartParseTask create OK\r\n");
236 else
237     printf("UartParseTask create FAIL\r\n");
238
239 ret = xTaskCreate(KeyTask,
240                  "KeyInput",
241                  256,
242                  NULL,
243                  tskIDLE_PRIORITY + 2,
244                  &keyTaskHandle);
245
246 if (ret == pdPASS)
247     printf("KeyTask create OK\r\n");
248 else
249     printf("KeyTask create FAIL\r\n");
250
251 ret = xTaskCreate(DisplayTask,
252                  "Display",
253                  512,
254                  NULL,
255                  tskIDLE_PRIORITY + 1,
256                  &displayTaskHandle);
257
258 if (ret == pdPASS)
259     printf("DisplayTask create OK\r\n");
260 else
261     printf("DisplayTask create FAIL\r\n");
```

```

185 BaseType_t ret;
186
187 requestQueueHandle = xQueueCreate(REQUEST_QUEUE_LEN, sizeof(RequestMsg_t));
188 uartRxQueueHandle = xQueueCreate(UART_RX_QUEUE_LEN, sizeof(uint8_t));
189
190 stateMutexHandle = xSemaphoreCreateMutex();
191 uartMutexHandle = xSemaphoreCreateMutex();
192
193 if (requestQueueHandle == NULL)
194     printf("requestQueue create FAIL\r\n");
195 else
196     printf("requestQueue create OK\r\n");
197
198 if (uartRxQueueHandle == NULL)
199     printf("uartRxQueue create FAIL\r\n");
200 else
201     printf("uartRxQueue create OK\r\n");
202
203 if (stateMutexHandle == NULL)
204     printf("stateMutex create FAIL\r\n");
205 else
206     printf("stateMutex create OK\r\n");
207
208 if (uartMutexHandle == NULL)
209     printf("uartMutex create FAIL\r\n");
210 else
211     printf("uartMutex create OK\r\n");

```

系统状态数据结构

系统使用 ElevatorState_t 集中保存当前楼层、方向、门状态、三类请求数组和累计服务次数。请求数组长度为 5，仅使用下标 1~4，使数组下标与楼层号直接对应，简化了查询和清除操作。

核心状态结构

```

typedef struct
{
    uint8_t currentFloor;
    ElevatorDir_t dir;
    DoorState_t door;
    uint8_t insideReq[5];
    uint8_t upReq[5];
    uint8_t downReq[5];
    uint32_t servedCount;
} ElevatorState_t;

```

请求表示与去重

所有输入最终转换为 RequestMsg_t，包含 floor 和 type 两个字段。Elevator_AddRequest 根据 type 将对应数组元素置 1。由于数组元素仅表示“有/无请求”，相同请求多次到达不会重复占用空间，也不会导致重复停靠，天然实现去重。

电梯控制任务流程

ElevatorTask 每轮首先处理新请求，然后在互斥保护下判断当前层是否需要停靠。如果需要，则释放互斥量后执行开门服务；否则决定方向并移动一层。无请求或方向为空闲时，任务阻塞等待请求队列 100 ms，避免空转占用 CPU。

按键任务流程

1. 等待外部中断发出的直接任务通知。
2. 延时 20 ms 进行按键消抖。
3. 设置 keyScanning 标志，逐列驱动并读取四条行线。
4. 仅接受键值 1~4，封装为 REQ_INTERNAL 消息发送到 requestQueue。
5. 等待按键释放，清空抖动期间积累的通知后重新阻塞。

串口解析任务流程

1. 永久等待 uartRxQueue 中的一个字节。
2. 将小写方向字符转换为大写，忽略空格、回车和换行。
3. 未保存楼层时，只接受字符‘1’~‘4’。
4. 已保存楼层时，接受‘U’或‘D’，构造外部请求并发送到 requestQueue。
5. 遇到非法字符时输出错误信息并复位解析状态。

显示任务流程

DisplayTask 每 2 s 执行一次 Display_PrintState。函数先复制共享状态，再在 uartMutex 保护下输出两行状态信息，最后调用 LCD_ShowElevatorState 刷新图形界面。LCD 界面分为三个信息卡片和三行请求列表，底部显示累计服务次数和串口速率。

6.1 FreeRTOS 对象初始化

MX_FREERTOS_Init 创建两个队列、两个互斥量和四个任务，并逐项输出创建结果。最后启动 USART1 单字节中断接收。该初始化顺序保证任务开始运行前，所需通信对象已经建立。

```
requestQueueHandle = xQueueCreate(16, sizeof(RequestMsg_t));
uartRxQueueHandle = xQueueCreate(32, sizeof(uint8_t));
stateMutexHandle = xSemaphoreCreateMutex();
uartMutexHandle = xSemaphoreCreateMutex();
```

矩阵键盘扫描

键盘列线平时全部置高，以便任一按键使对应行线产生高电平并触发 EXTI。进入 KeyTask 后，先将所有列拉低，再逐列置高并读取四条行线，根据 keyMap 映射得到键值。扫描期间 keyScanning 置 1，外部中断回调检查该标志，避免扫描自身引起重复通知。

请求合法性检查

Elevator_AddRequest 首先检查楼层是否在 1~4 范围内，并拒绝 1D 和 4U。合法请求在 stateMutex 保护下写入相应数组。由于数组置位操作具有去重效果，重复输入只会保持该位为 1。

```
if (msg->type == REQ_EXTERNAL_DOWN && msg->floor == 1) valid = 0;  
if (msg->type == REQ_EXTERNAL_UP && msg->floor == 4) valid = 0;
```

逐层移动

Elevator_MoveOneFloor 根据方向计算 toFloor，先输出移动日志，再执行 3 s 分段延时，最后更新 currentFloor。楼层只在延时结束后改变，因此显示结果能够区分“正在从某层移动”和“已到达下一层”。边界判断保证上行不会超过 4 层，下行不会低于 1 层。

到站服务与开关门

到站后，函数将门状态置为 DOOR_OPEN，清除当前楼层三类请求并增加 servedCount。门保持期间仍调用 Elevator_DelayWithInput 接收新请求，延时结束后置门状态为 DOOR_CLOSED。

串口与 LCD 状态显示

串口输出包括事件日志和周期状态。周期状态格式示例如下：

```
[STATE] floor=3F, dir=UP, door=CLOSED, served=2  
IN: 4F | UP: none | DOWN: 2F
```

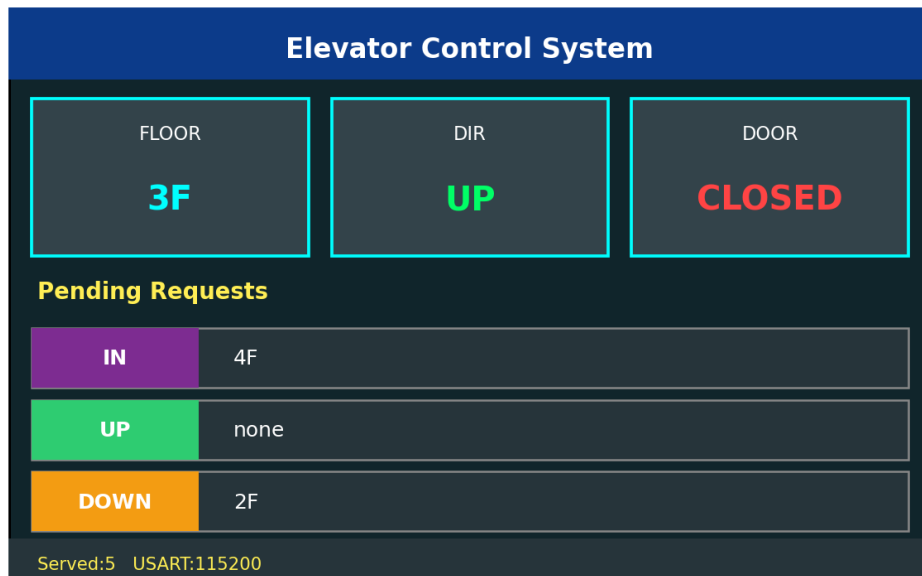
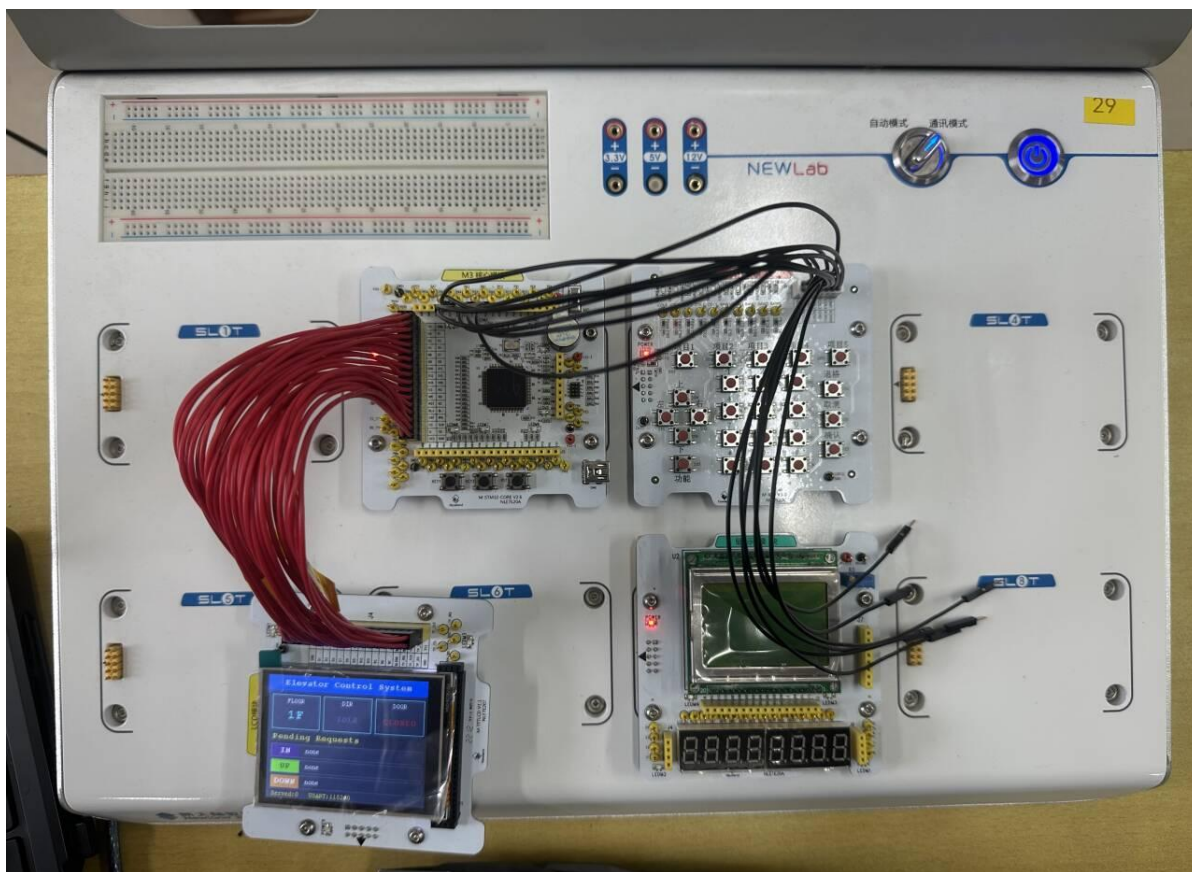


图 6-1 LCD 界面示意图

源码实现特点

1. ISR 只执行通知、入队和重新启动接收，符合短中断原则。
2. 统一请求消息降低了输入设备与调度模块之间的耦合。
3. 请求位图结构简单、空间固定，适合四层小规模系统。
4. 显示任务使用状态快照，临界区短。
5. 移动和开门使用分段延时，保持新请求响应能力。
6. 非法边界请求和非法串口字符均有提示。

接线图



测试用例

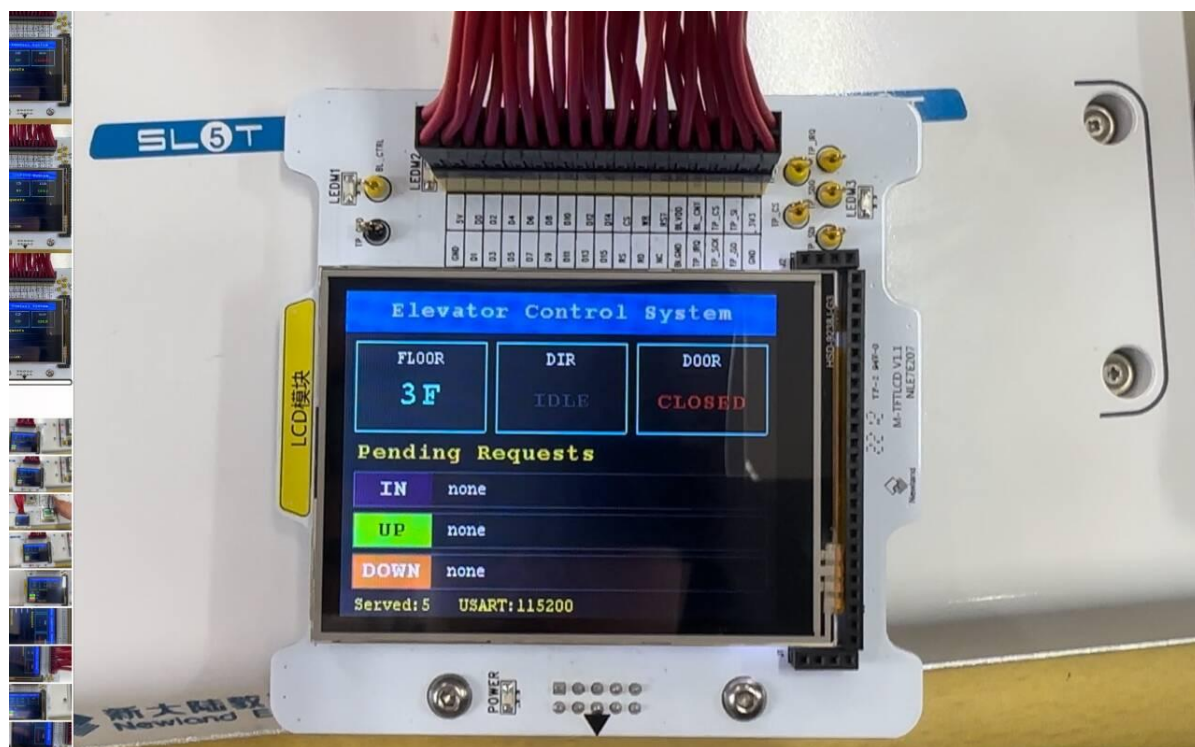
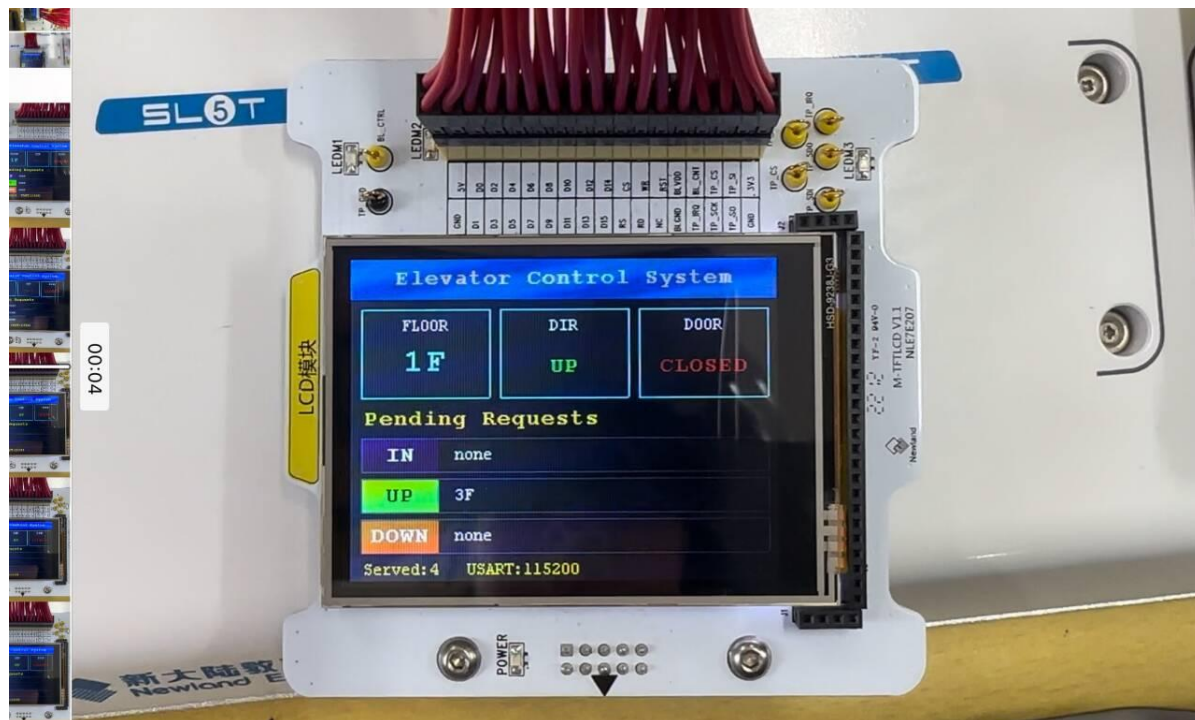
测试用例 1：系统上电初始化显示

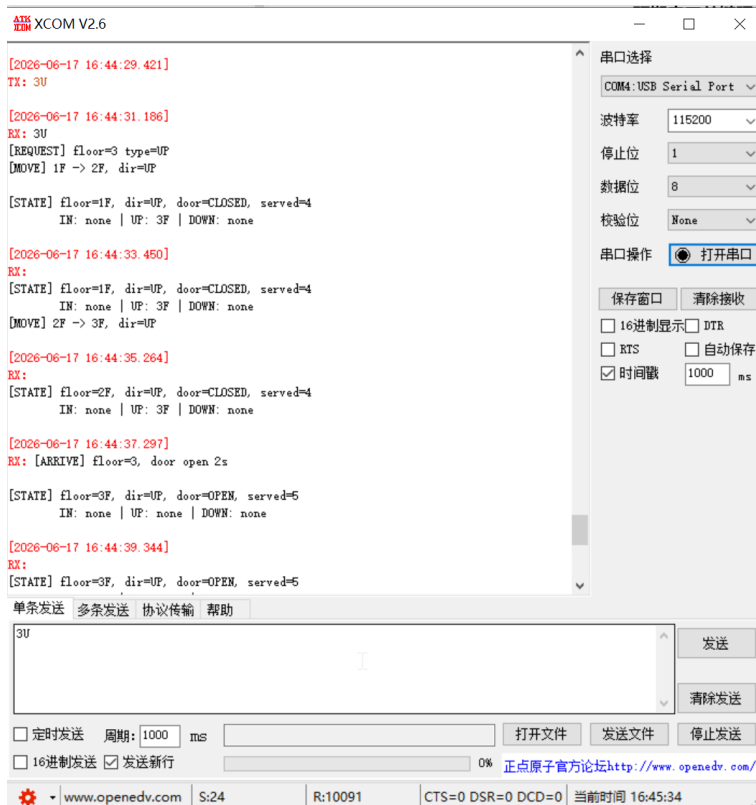
测试用例 2：单个外部上行呼叫

测试目的：验证串口外部上行呼叫指令可被正确解析，并加入电梯待响应请求队列。

初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：串口发送指令：3U。



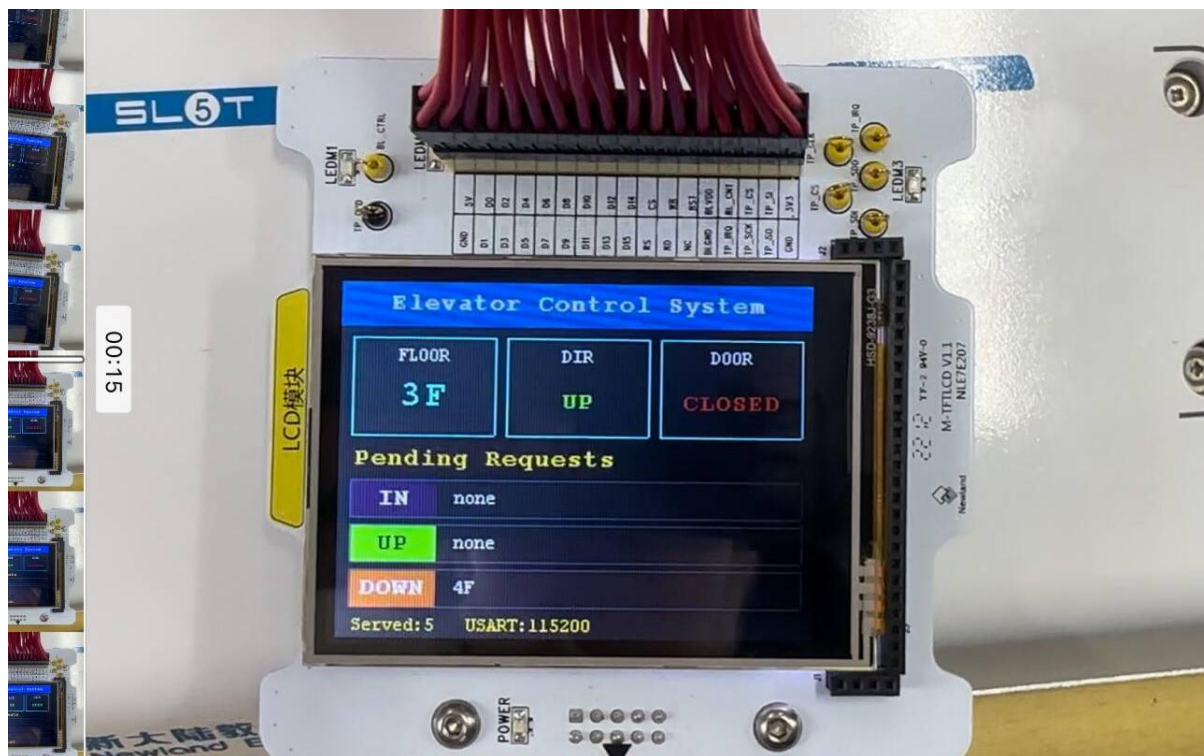


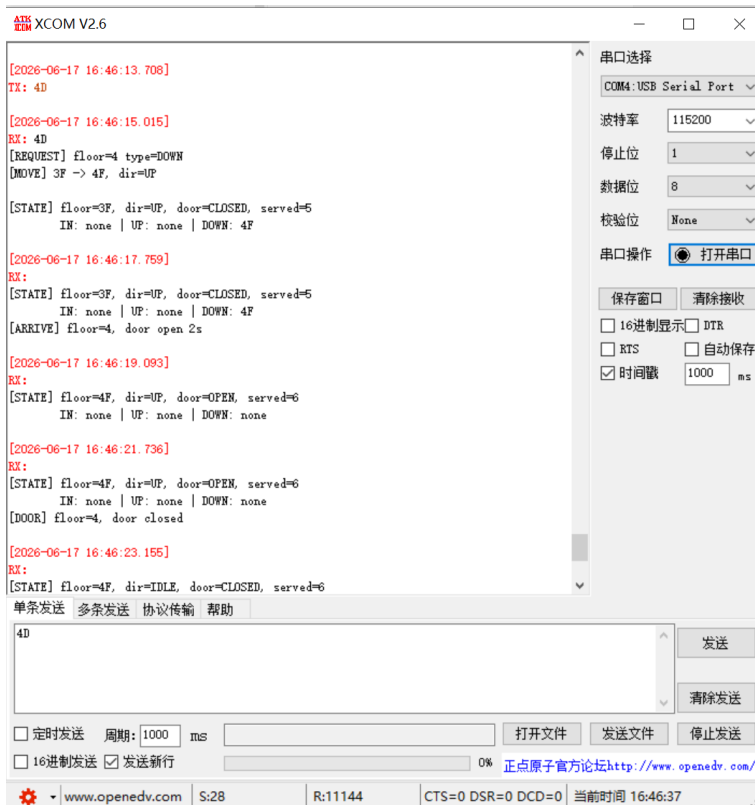
测试用例 3：单个外部下行呼叫

测试目的：验证外部下行呼叫指令能够被系统正确响应调度。

初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：串口发送指令：4D。

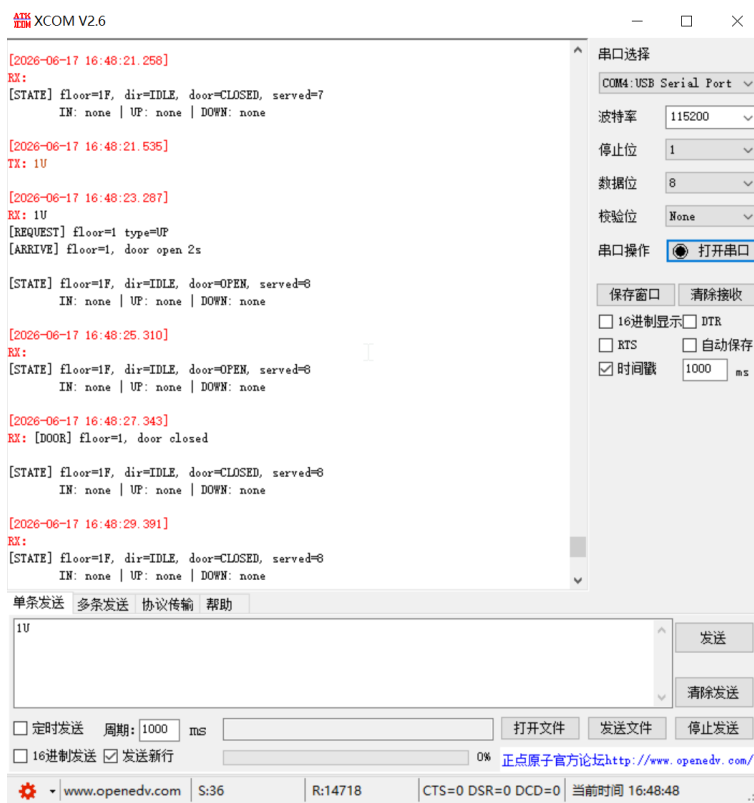




测试用例 4：当前楼层外部呼叫

测试目的：验证当前楼层存在呼梯请求时，电梯不移动，直接开门提供服务。

初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。



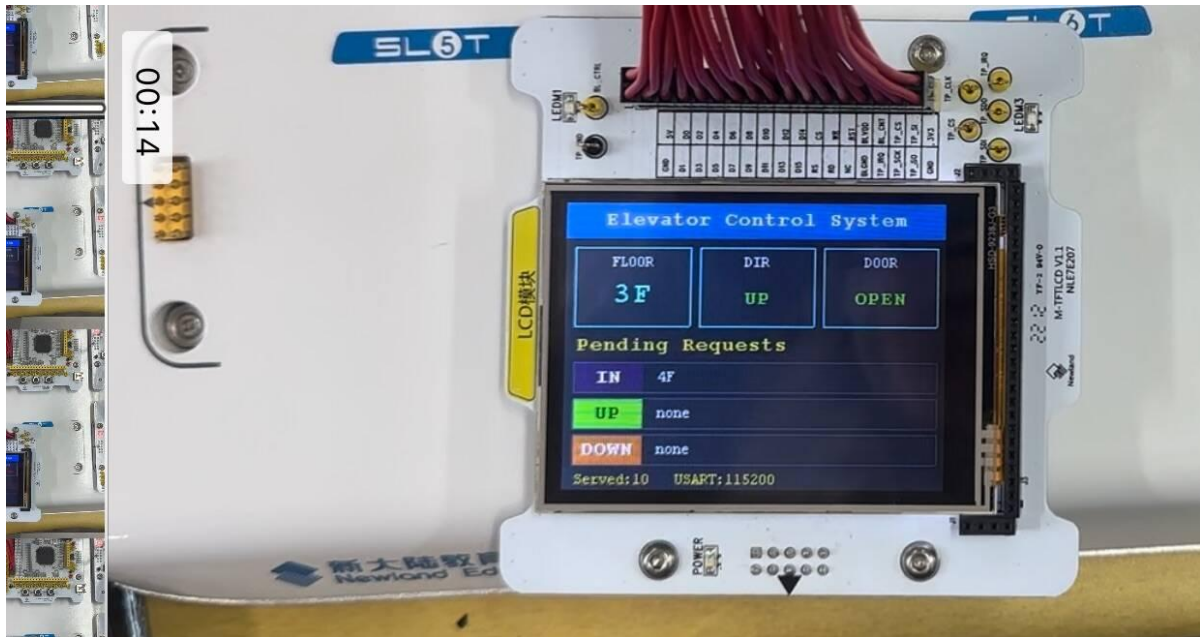
测试用例 5：内部楼层按钮请求

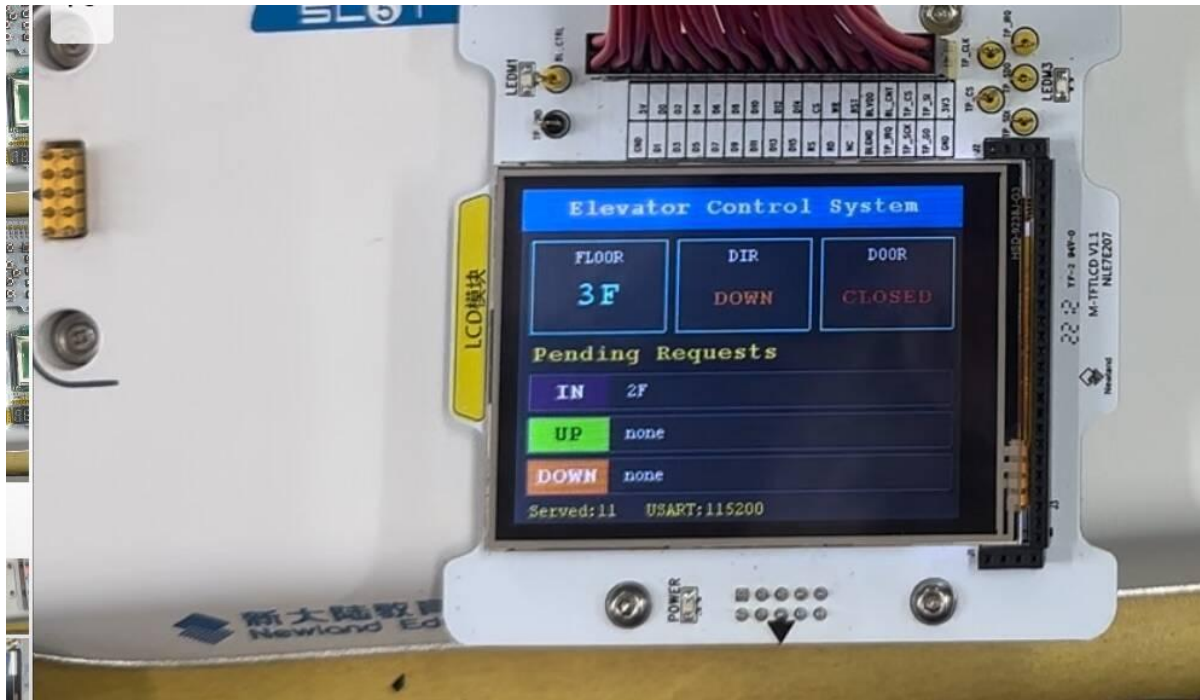
测试目的：验证轿厢内部按键数字 1~4，可以解析为轿厢内目标楼层乘梯请求。

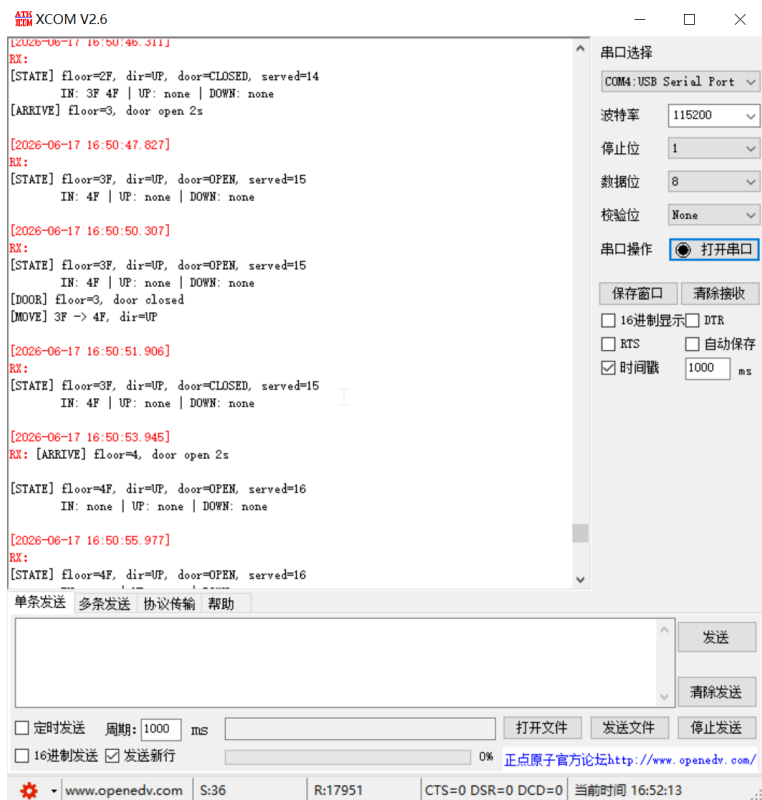
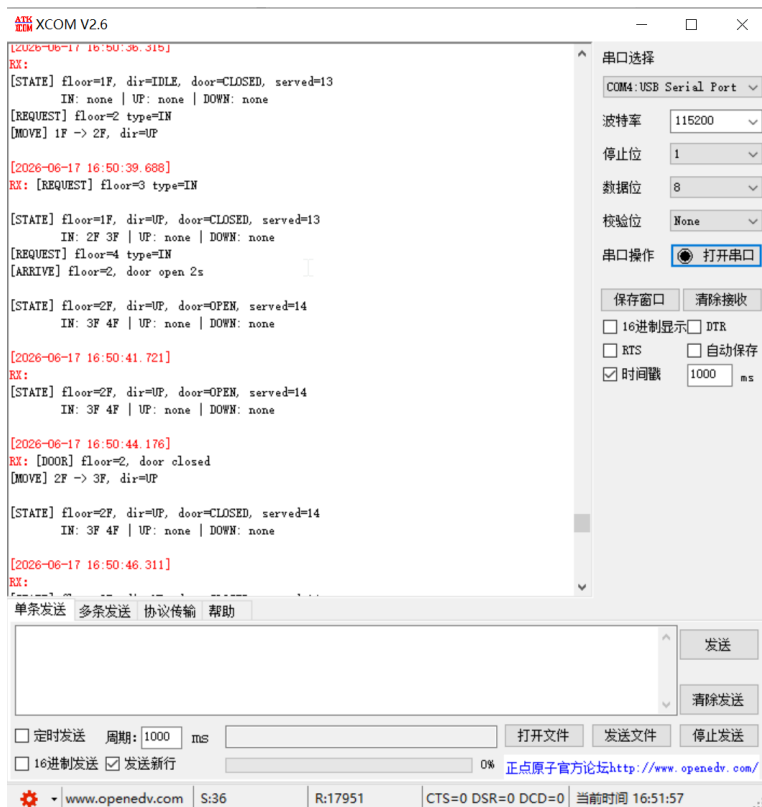
初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：按矩阵键盘 1、2、3、4，到 4 楼后，再按 2。









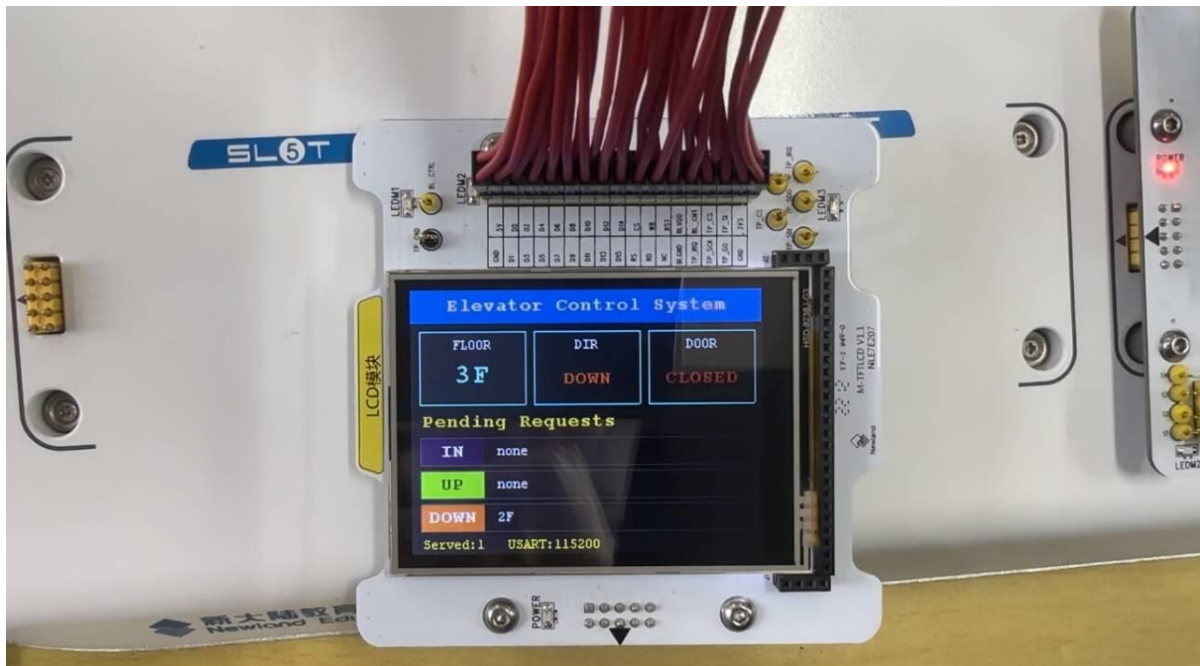
测试用例 6：同向优先测试一（上行时优先响应上行请求）

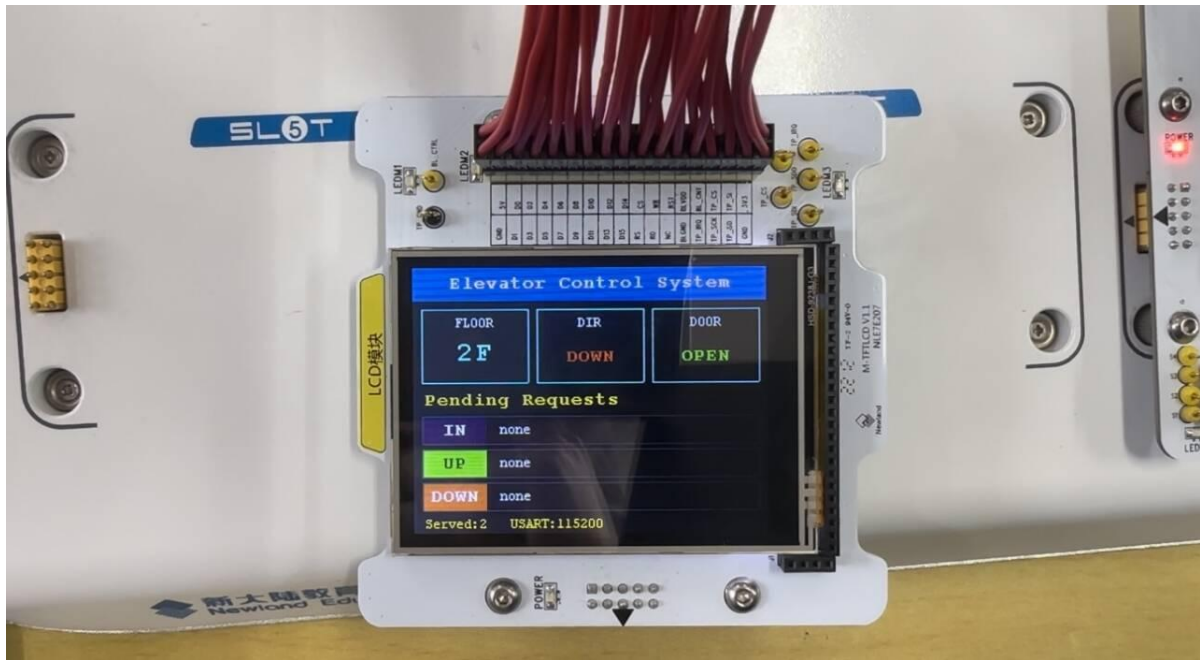
测试目的：验证电梯当前运行方向下，优先响应同向顺向呼梯请求。

初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：快速，几乎同时地，依次发送两条指令：2D、3U。







```

ATK XCOM V2.6
[2026-06-17 16:53:11.369]
RX:
[STATE] floor=1F, dir=IDLE, door=CLOSED, served=0
IN: none | UP: none | DOWN: none

[2026-06-17 16:53:13.371]
TX: 2D
3U

[2026-06-17 16:53:15.444]
RX:
[STATE] floor=1F, dir=IDLE, door=CLOSED, served=0
IN: none | UP: none | DOWN: none
2D
3U
[REQUEST] floor=2 type=DOWN
[MOVE] 1F -> 2F, dir=UP
[REQUEST] floor=3 type=UP

[STATE] floor=1F, dir=UP, door=CLOSED, served=0
IN: none | UP: 3F | DOWN: 2F

[2026-06-17 16:53:17.480]
RX: [MOVE] 2F -> 3F, dir=UP

[STATE] floor=2F, dir=UP, door=CLOSED, served=0
IN: none | UP: 3F | DOWN: 2F

[2026-06-17 16:53:20.397]
RX:
[STATE] floor=2F, dir=UP, door=CLOSED, served=0
IN: none | UP: 3F | DOWN: 2F

```

串口选择: COM4: USB Serial Port

波特率: 115200

停止位: 1

数据位: 8

校验位: None

串口操作: ☒ 打开串口

保存窗口 清除接收

☐ 16进制显示 ☐ DTR

☐ RTS ☐ 自动保存

☒ 时间戳 1000 ms

单条发送 多条发送 协议传输 帮助

2D
3U

发送

清除发送

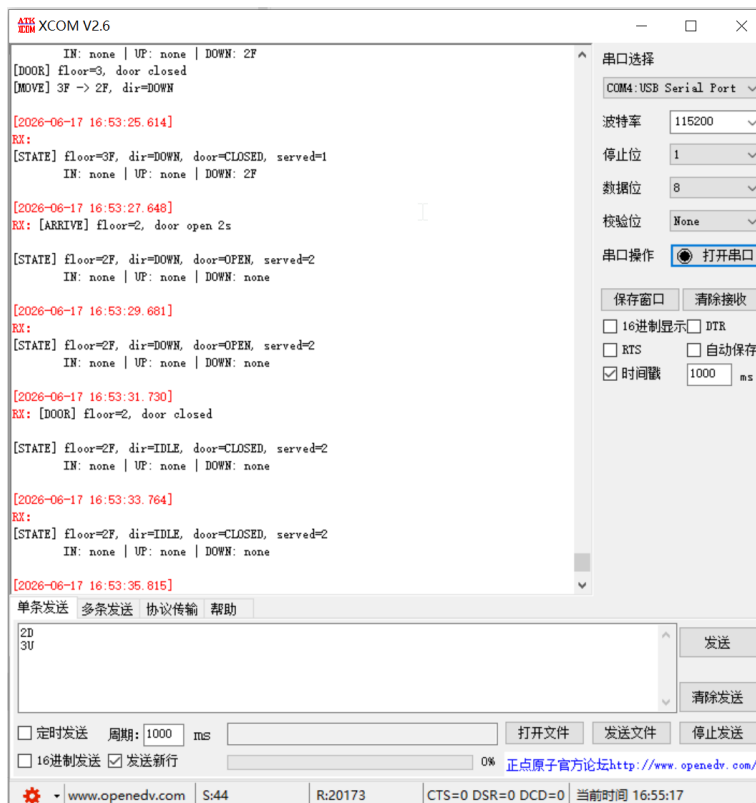
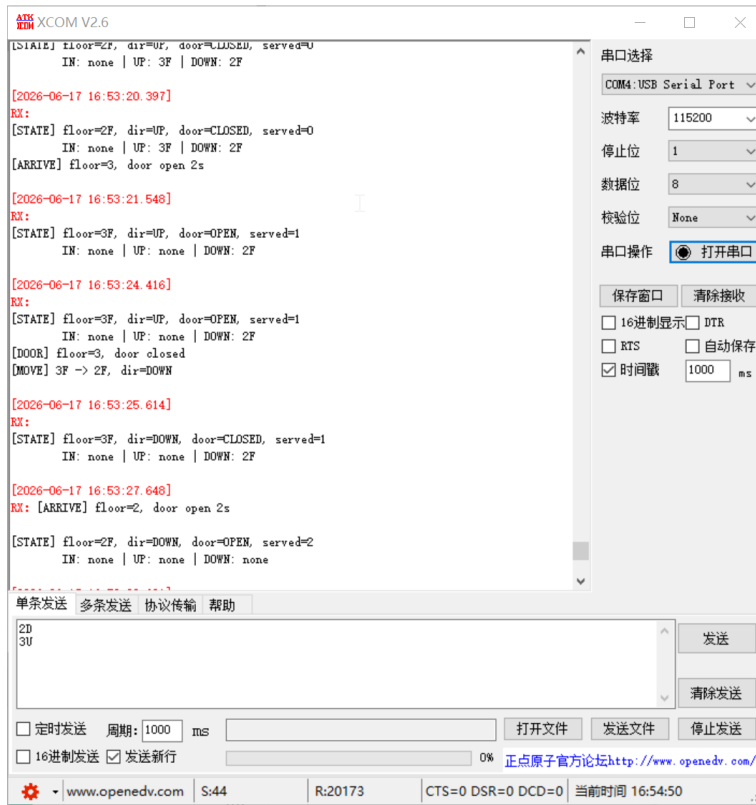
☐ 定时发送 周期: 1000 ms

☐ 16进制发送 ☒ 发送新行

打开文件 发送文件 停止发送

0% 正点原子官方论坛 <http://www.openedv.com/>

www.openedv.com S:44 R:20173 CTS=0 DSR=0 DCD=0 当前时间 16:54:07

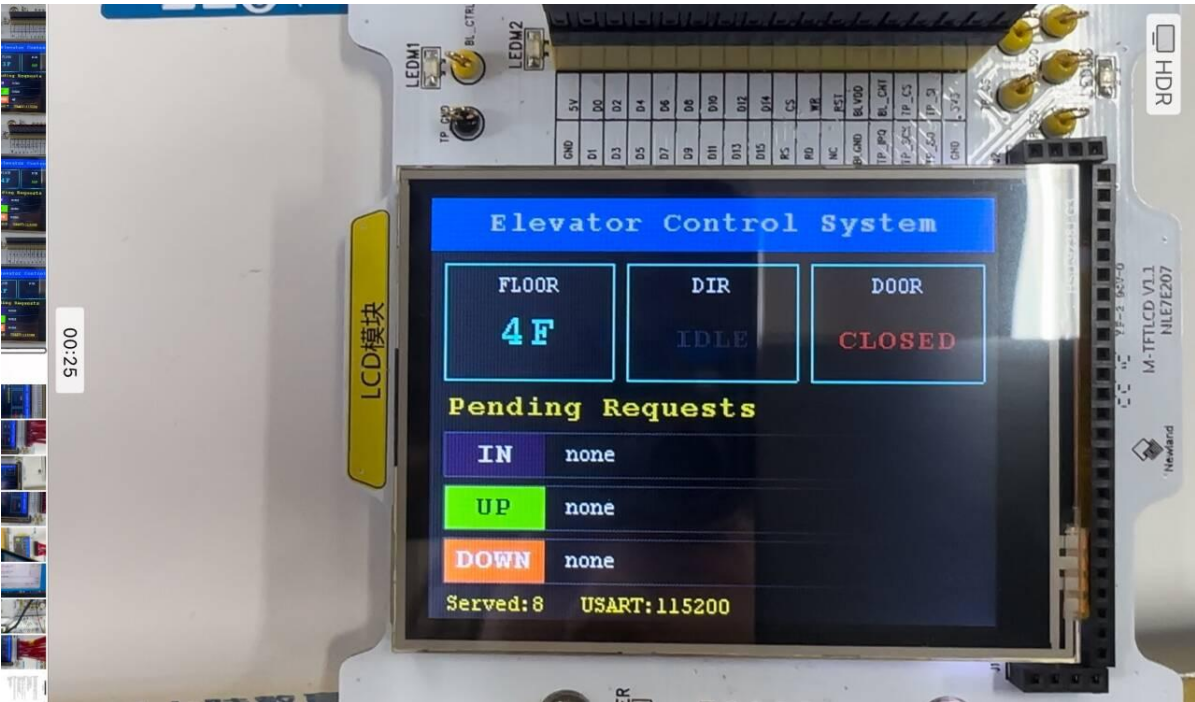
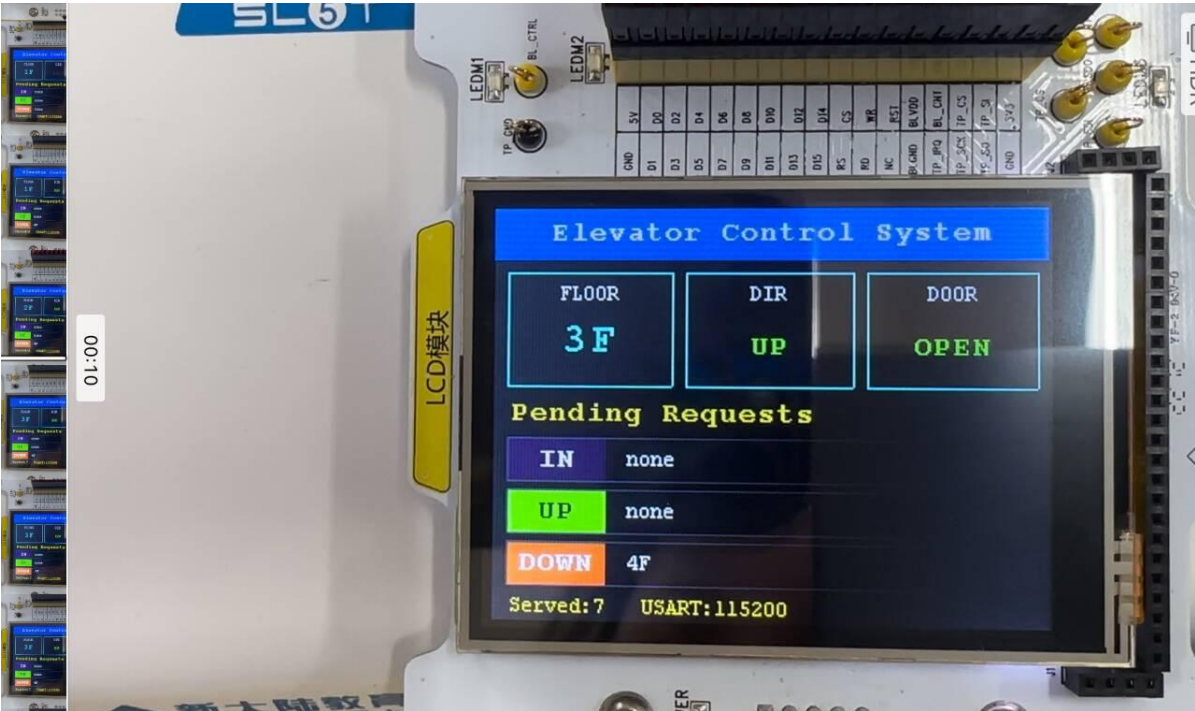


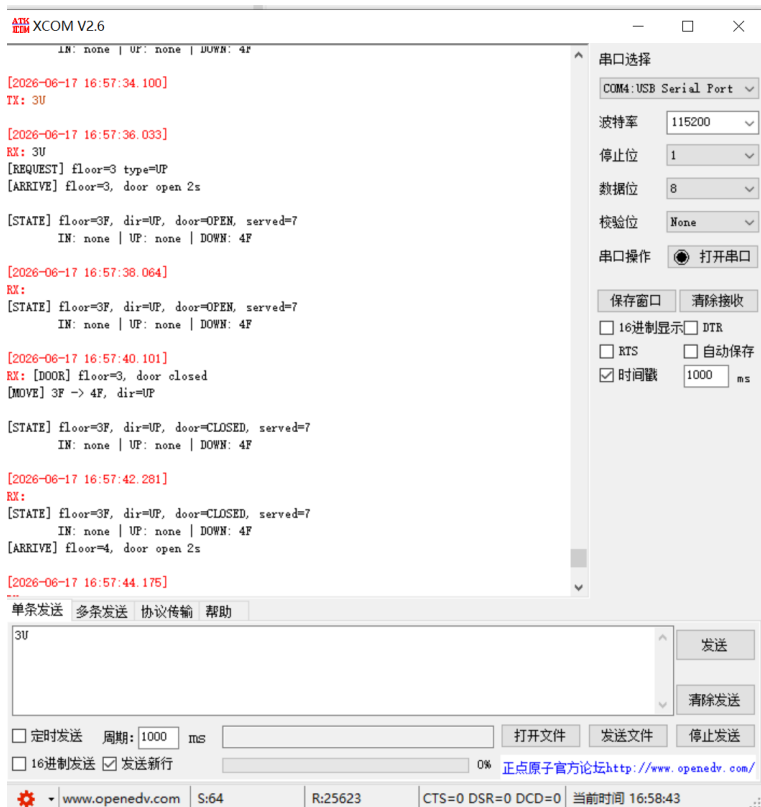
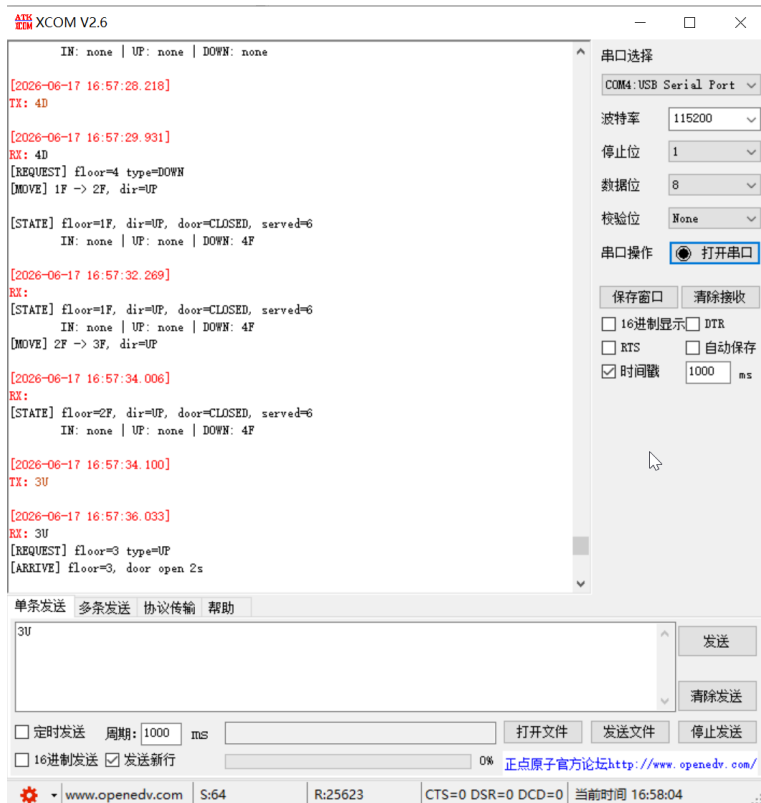
测试用例 7：运行过程中新增同向请求

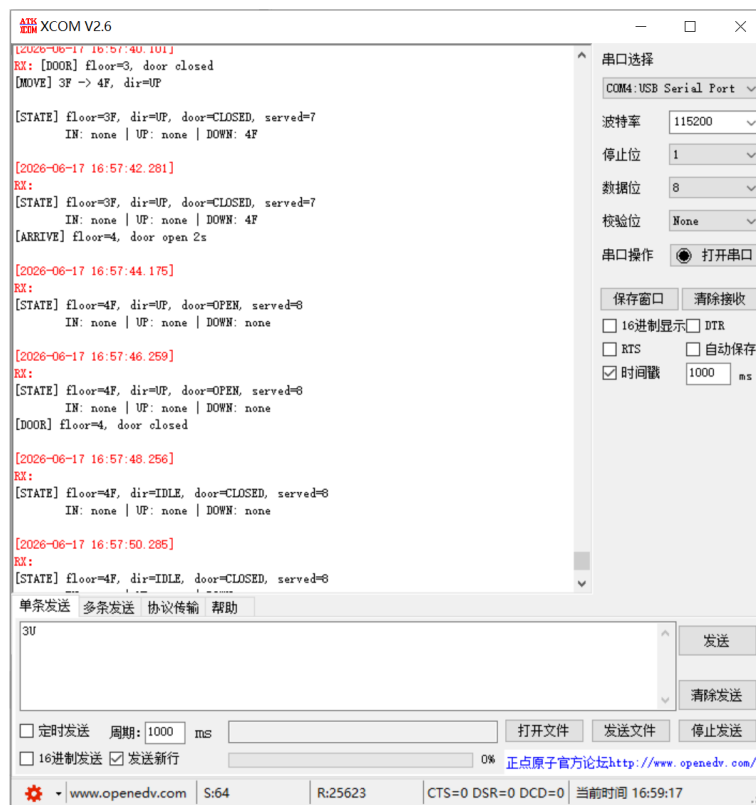
测试目的：验证电梯运行途中可以实时接收新呼梯请求，并顺路响应同向排队请求。

初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：先发指令 4D；电梯开始上行、尚未抵达 2 层时，补发指令 3U。







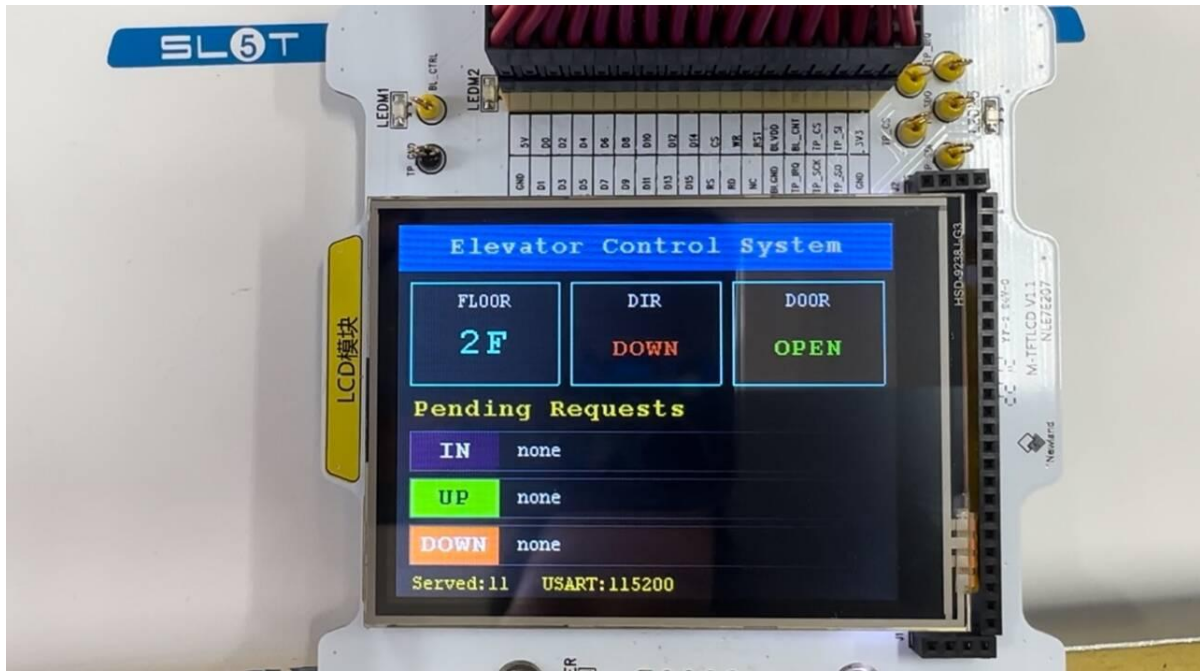
测试用例 8：运行过程中新增反向请求

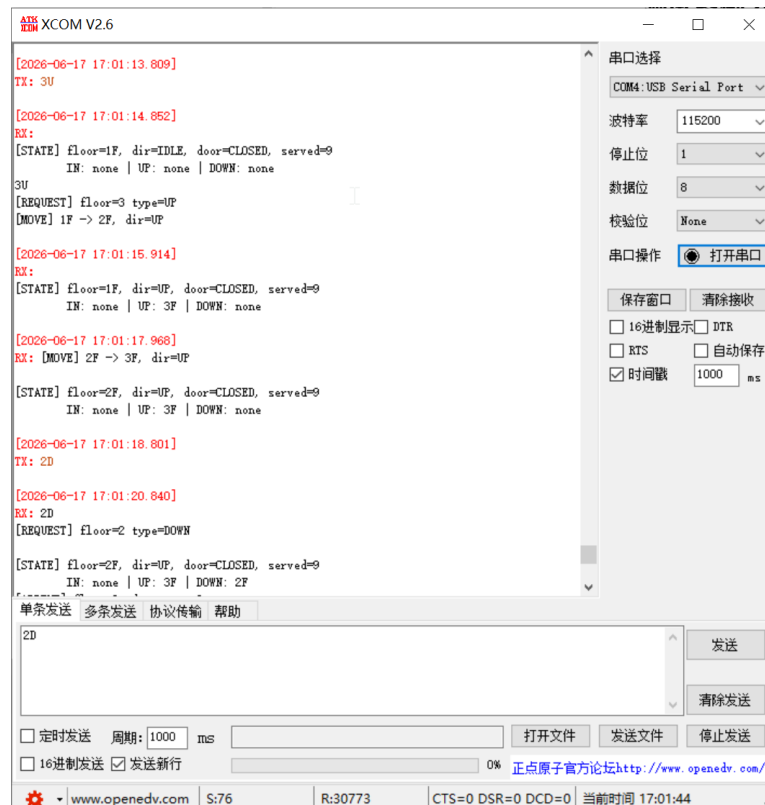
测试目的：验证电梯运行途中收到反向呼梯时，不会中途换向插队，等当前方向所有顺路请求全部处理完毕后，再反向处理反向排队请求。

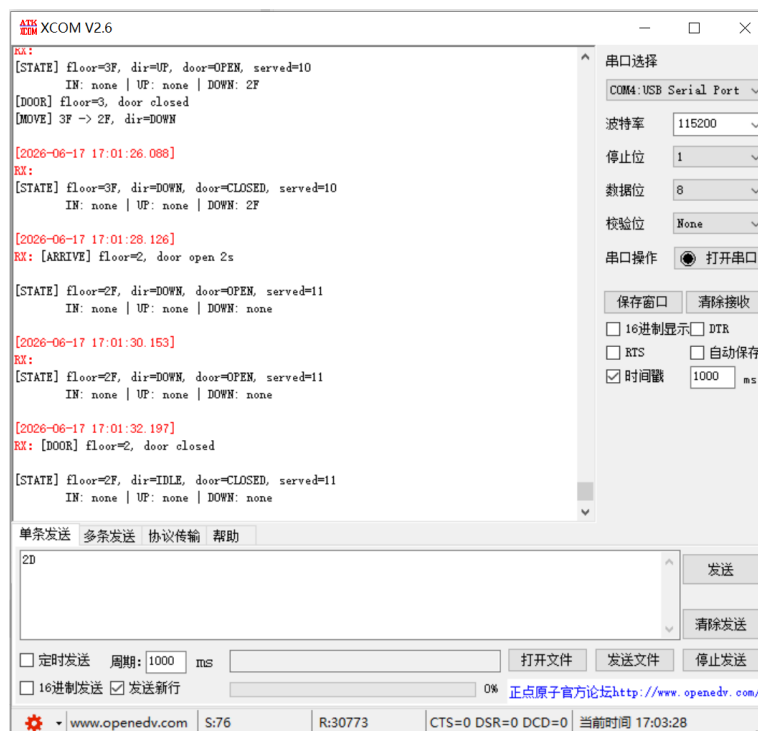
初始状态：电梯位于 1 层，轿厢门关闭，待机空闲。

操作步骤：先发指令 3U；电梯上行启动后，补发指令 2D。







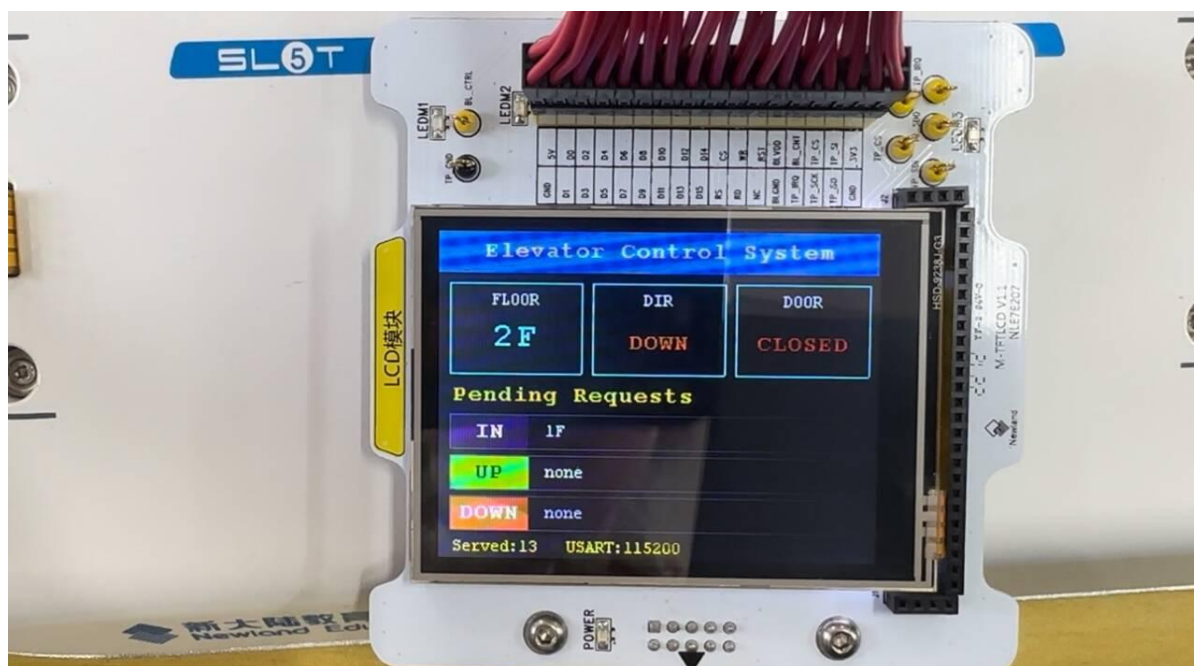


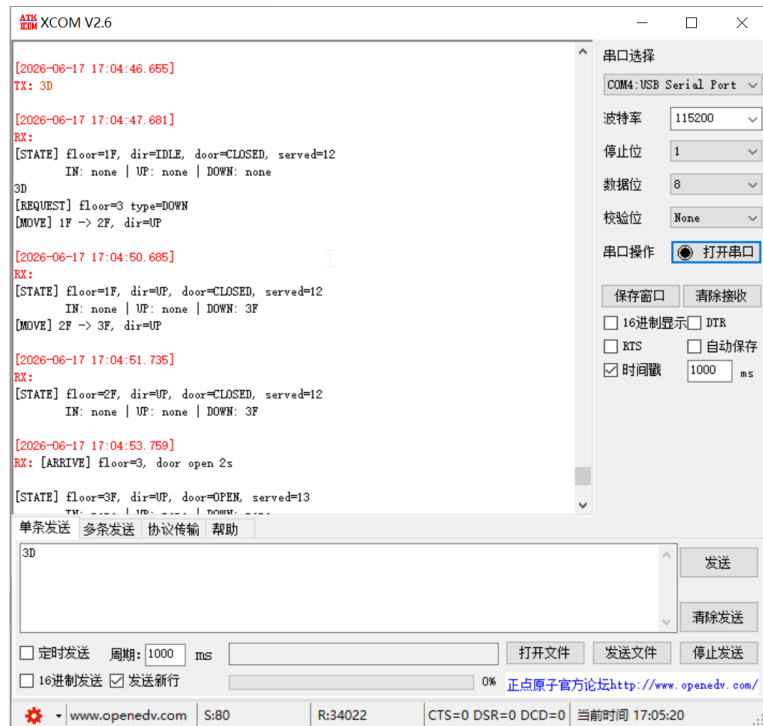
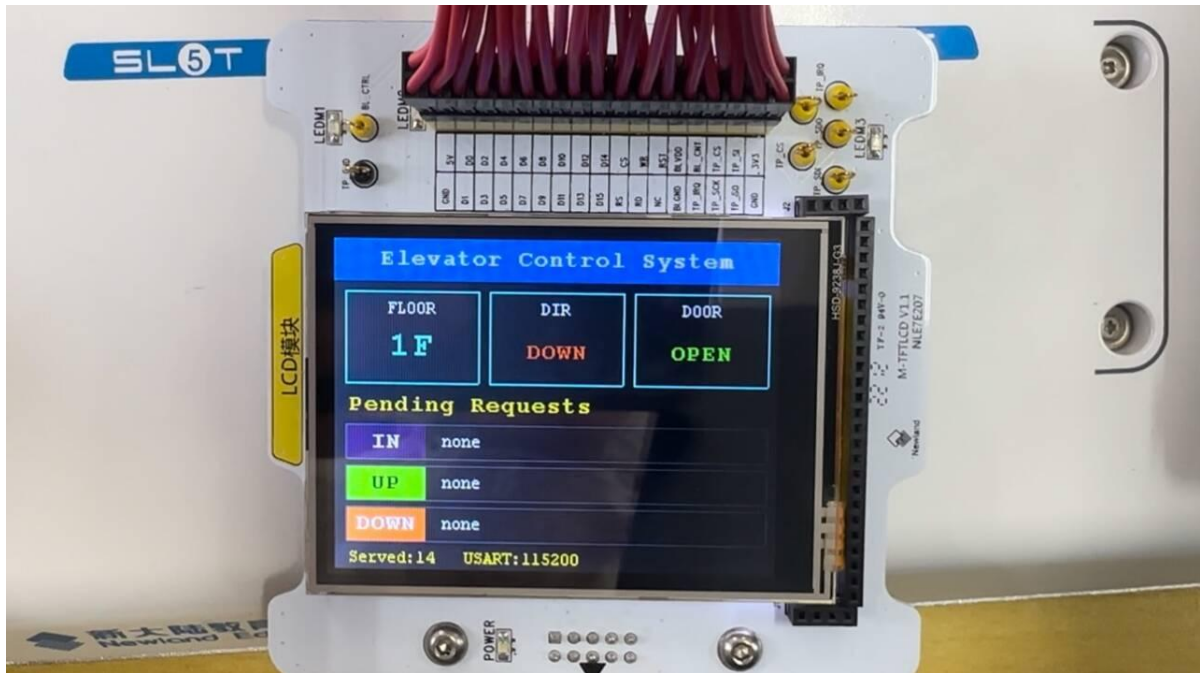
测试用例 9：键盘按键外部上行呼叫

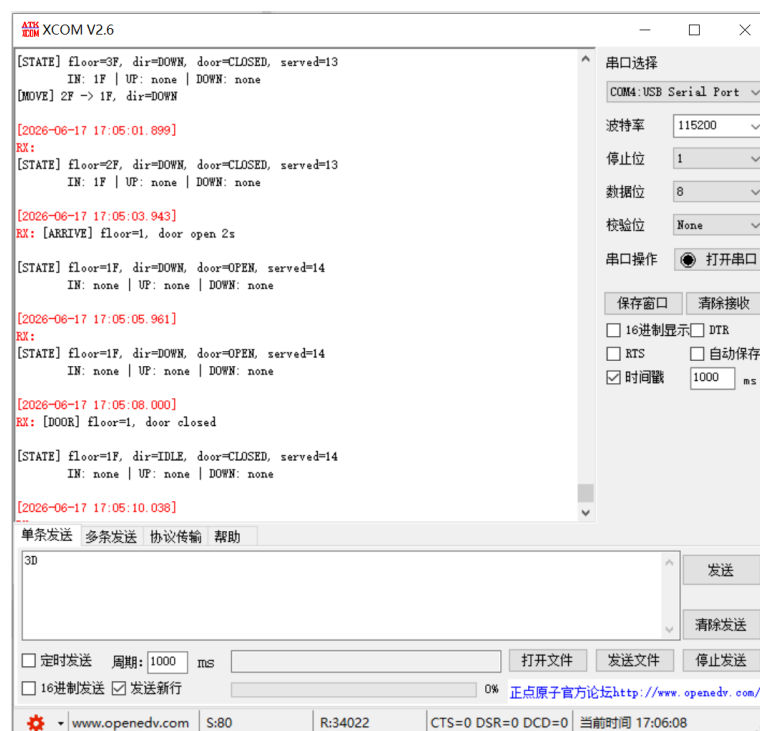
测试目的：验证开发板物理键盘 / 按键可以模拟外部上行呼叫，并被系统正确登记与调度。

初始状态：电梯位于 3 层，轿厢门关闭，系统处于待机状态。

操作步骤：按下 1 楼的按键，下行到 1 楼。





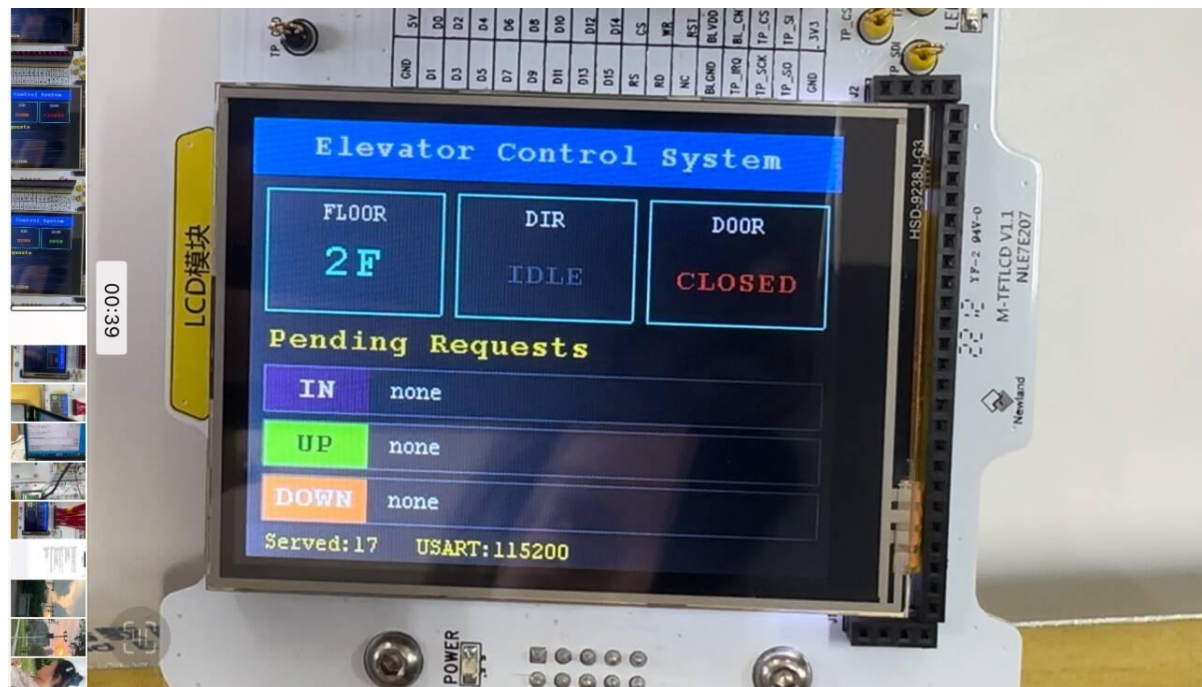
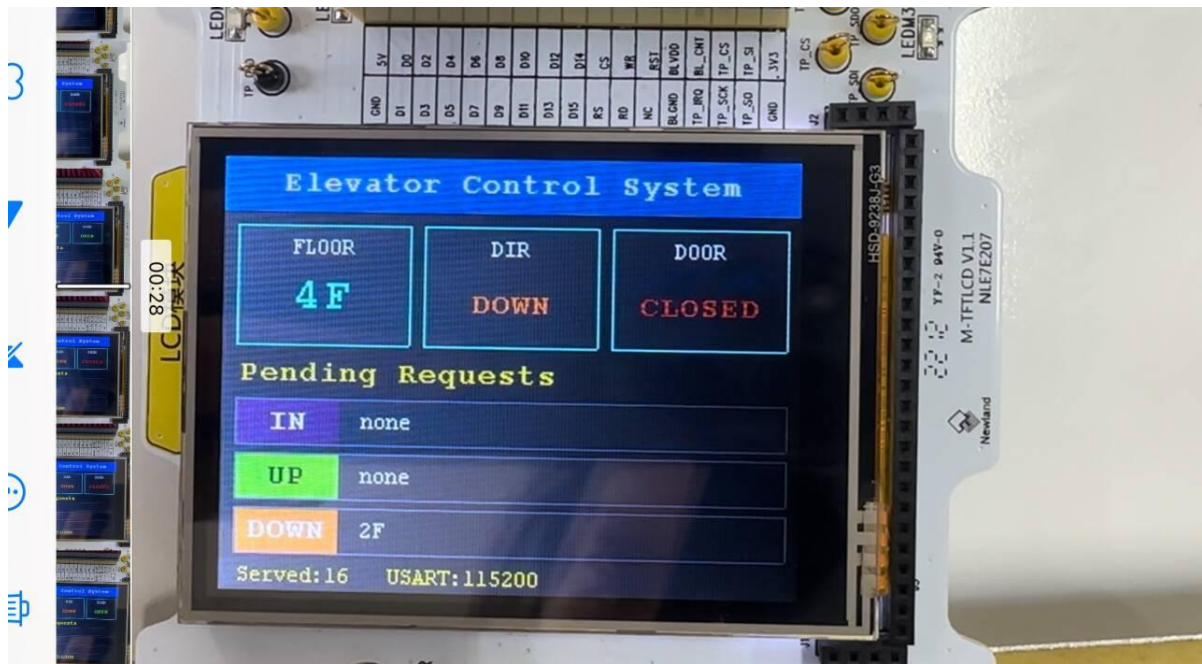


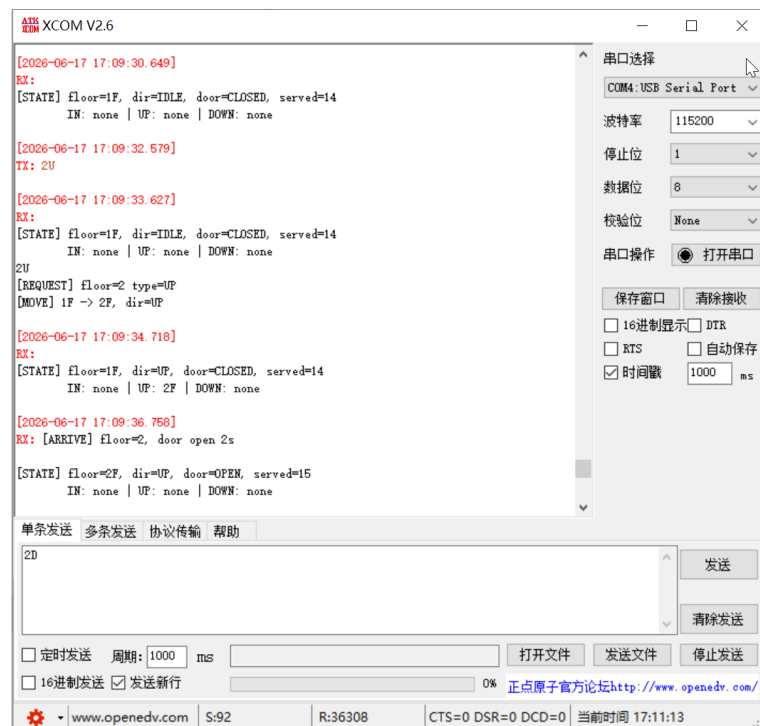
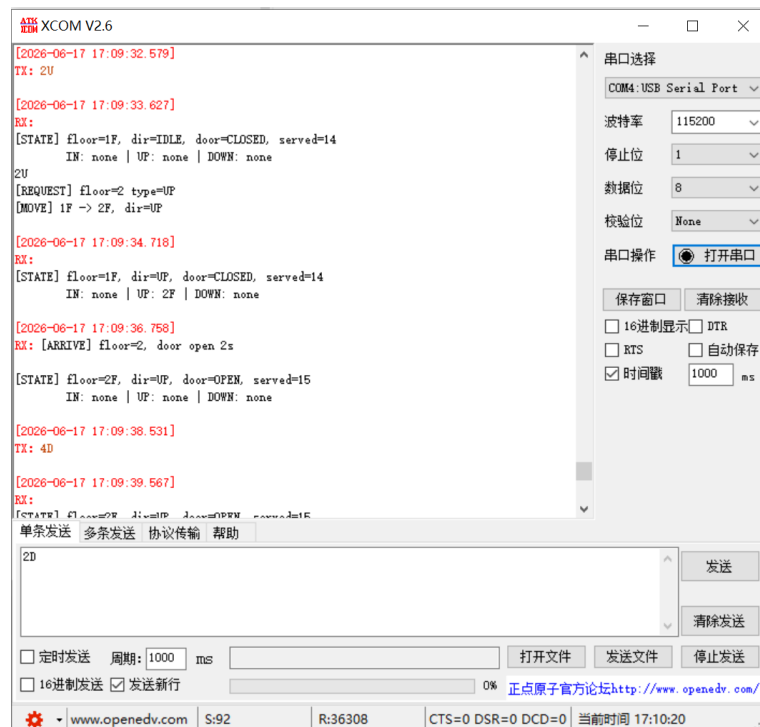
测试用例 10：串口指令与键盘按键混合测试

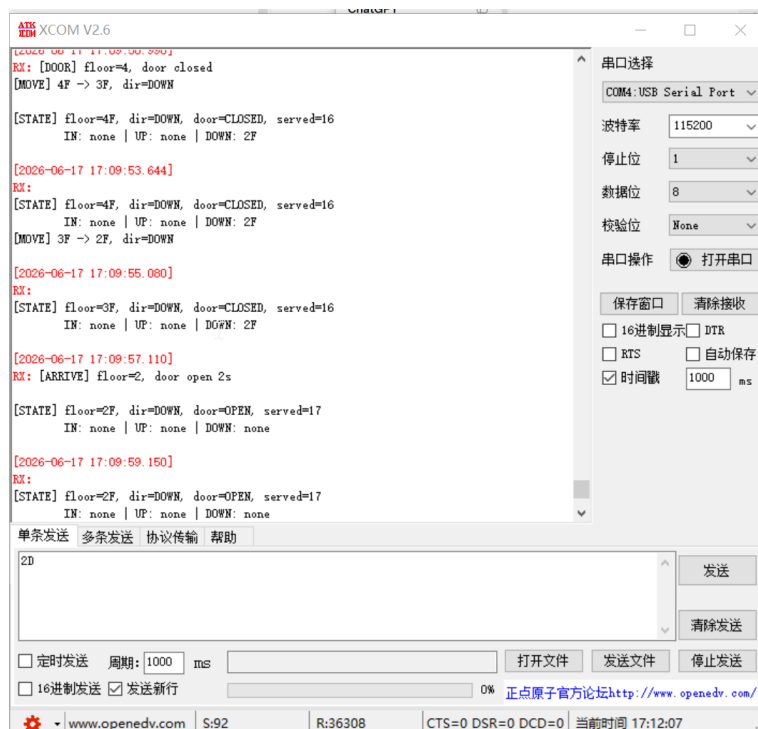
测试目的：验证系统可以同时响应串口指令和键盘按键输入，两种输入方式不会冲突。

初始状态：电梯位于 1 层，轿厢门关闭，系统处于待机状态。

操作步骤：串口发送 2U；在电梯运行过程中，按下键盘 4 楼内部按钮；串口发送 2D。







测试用例 11：边界楼层合法请求与非法外部请求过滤测试

测试目的：验证电梯一楼仅允许上行外呼、四楼仅允许下行外呼的边界规则；同时系统可识别非法呼梯指令，自动过滤无效请求、不加入待响应队列。

初始状态：电梯位于 1 层，轿厢门关闭，处于空闲待机状态

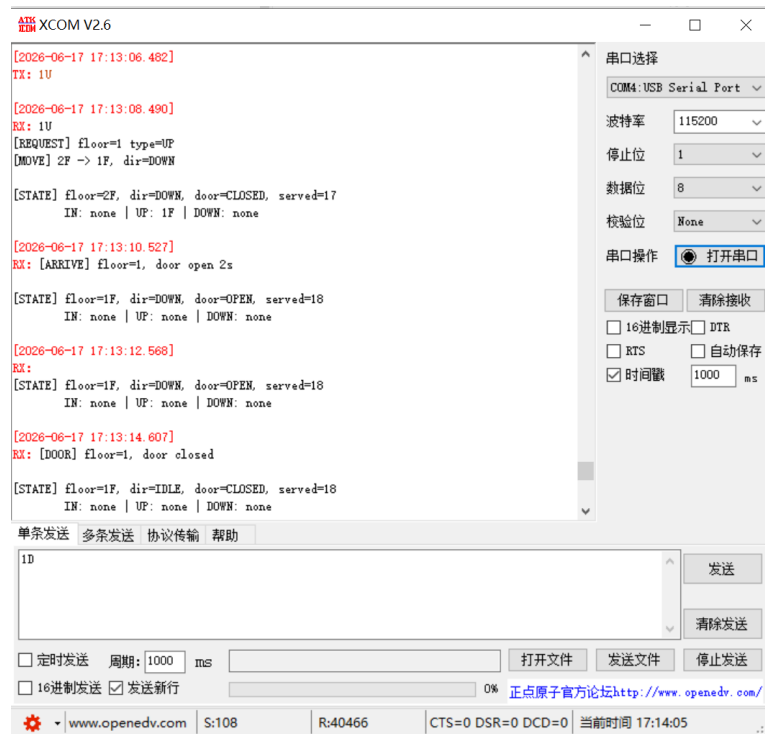
操作 A（合法边界指令测试）

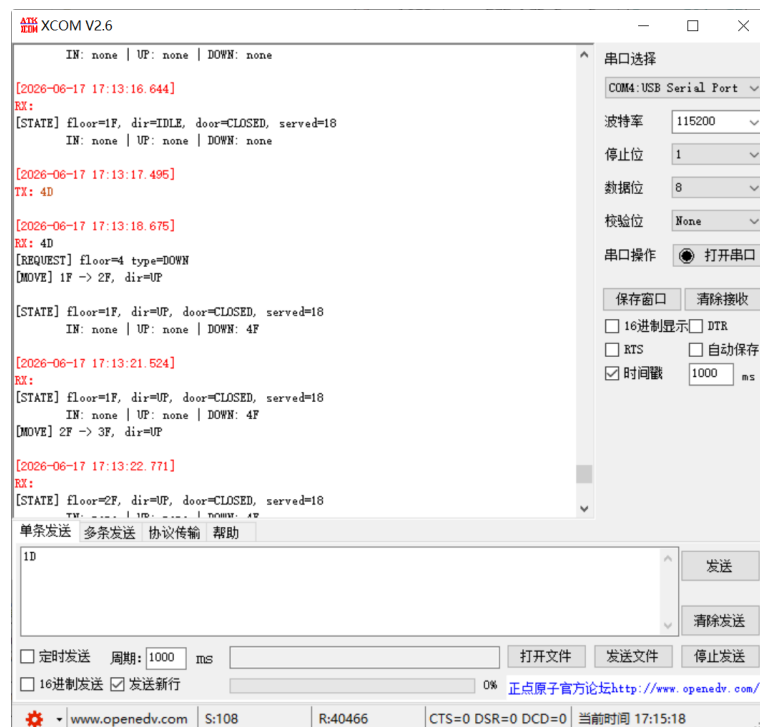
1. 上电复位后串口发送指令：1U
2. 电梯运行完成后，再次复位发送指令：4D

指令含义

1U：1 楼乘客上行外呼（一楼唯一合法外呼指令）

4D：4 楼乘客下行外呼（四楼唯一合法外呼指令）





操作 B（非法边界指令过滤测试）

1. 上电复位后串口发送指令：4U
2. 紧接着串口发送指令：1D

指令含义

1D: 1 楼下行外呼，一楼不存在向下楼层，属于非法请求

4U: 4 楼上行外呼，四楼为最高楼层，不存在向上楼层，属于非法请求

